

Assignment 14

**EBS vs S3: Choosing the Right Storage Option — Key Differences and Use Cases.
Attach and Manage EBS Volume for an EC2 Instance**

Prepared By

Shreya Panchal

AMAZON WEB SERVICES • EC2 • EBS • S3

1. Create a table comparing Amazon EBS and Amazon S3, listing at least 4 key differences (such as data type, access method, use case, and scalability) and give one real-world app example where each would be the better choice.

Answer

Feature	Amazon EBS (Elastic Block Store)	Amazon S3 (Simple Storage Service)
Data Type	Block Storage	Object Storage
Access Method	Attached directly to an EC2 instance as a virtual disk	Accessed through the internet using APIs, SDKs, or the AWS Management Console
Primary Use Case	Operating systems, databases, applications, and boot volumes	Images, videos, backups, documents, logs, and static website files
Scalability	Can be resized, but attached to specific EC2 instances	Virtually unlimited storage with automatic scaling
Performance	Low latency and high IOPS, suitable for frequently accessed data	Optimized for high durability and scalable object storage rather than low-latency block access
Availability	Available within a single Availability Zone (AZ)	Data is redundantly stored across multiple Availability Zones within a region
Real-World Example	MySQL database running on an Amazon EC2 instance for an online banking application	Instagram profile pictures and user-uploaded images , where large numbers of image files need scalable, durable storage

2. Launch a new EC2 instance in the AWS Free Tier and attach a new EBS volume of at least 2 GB to it using the AWS Management Console.
Hint: After launching the instance, use the 'Volumes' section in EC2 to create and attach the volume.

Answer

A new **Amazon EC2 Free Tier instance** was launched successfully, and a new **2 GB Amazon EBS (gp3)** volume was created and attached using the AWS Management Console.

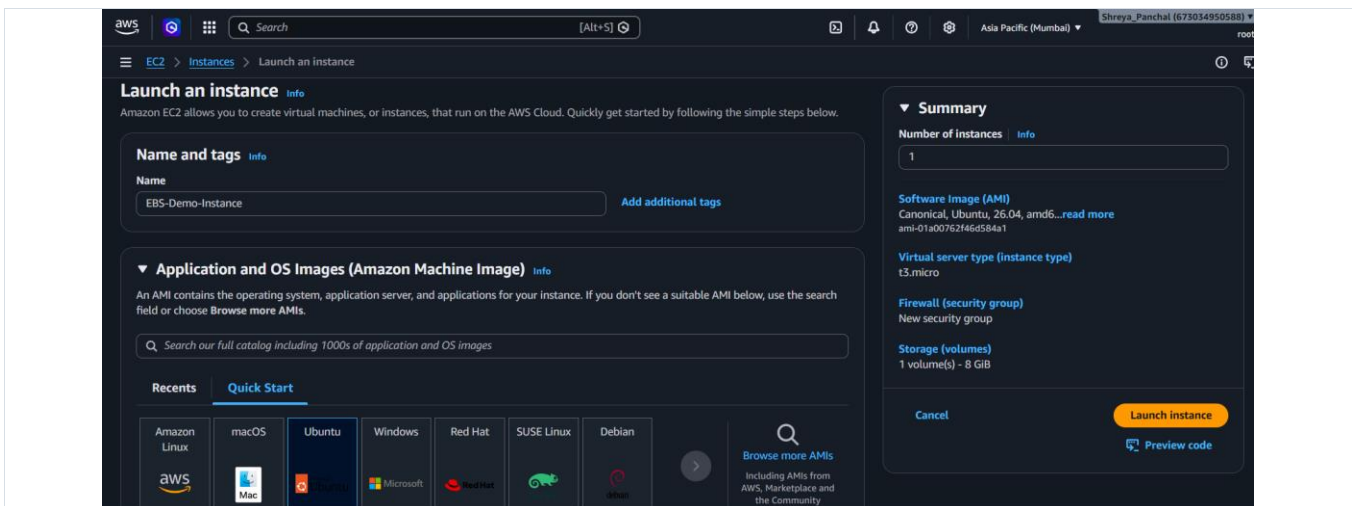
Steps Performed

1. Logged in to the AWS Management Console.
2. Opened the Amazon EC2 service.

3. Clicked Launch Instance.
4. Entered the instance name EBS-Demo-Instance.
5. Selected Amazon Linux 2023 AMI.
6. Chose the t2.micro instance type (Free Tier Eligible).
7. Selected an existing key pair.
8. Enabled Auto-assign Public IP.
9. Launched the EC2 instance.
10. Opened Elastic Block Store → Volumes.
11. Created a new gp3 EBS volume with a size of 2 GiB in the same Availability Zone as the EC2 instance.
12. Selected the new volume and clicked Actions → Attach Volume.
13. Selected the EC2 instance and attached the volume using the device name /dev/sdf.
14. Verified that the EBS volume status changed to In-use.

Result

The EC2 instance was launched successfully, and the new **2 GB EBS volume** was attached successfully. The volume is now available for formatting, mounting, and storing data.



Instances (1/1) Info

Save filter sets
Choose filter set

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS
EBS-Demo-Ins...	i-0843a873d317acb32	Running	t3.micro	3/3 checks passed	ap-south-1a	ec2-13-201-115-43.ap-...

i-0843a873d317acb32 (EBS-Demo-Instance)

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Instance summary Info

Instance ID i-0843a873d317acb32	Public IPv4 address 13.201.115.43 open address	Private IPv4 addresses 172.31.45.216
IPv6 address -	Instance state Running	Public DNS ec2-13-201-115-43.ap-south-1.compute.amazonaws.com

Attach volume Info

Attach a volume to an instance to use it as you would a regular physical hard disk drive.

Basic details

Volume ID
vol-026665b0f18f1e10e

Availability Zone
aps1-az1 (ap-south-1a)

Instance
i-0843a873d317acb32
(EBS-Demo-Instance) (running)

Only instances in the same Availability Zone as the selected volume are displayed.

Device name
/dev/sdb

Recommended device names for Linux: /dev/sda1 for root volume, /dev/sd[f-p] for data volumes.

ⓘ Newer Linux kernels may rename your devices to /dev/xvdf through /dev/xvdp internally, even when the device name entered here (and shown in the details) is /dev/sdf through /dev/sdp.

Storage

Details | Status and alarms | Monitoring | Security | Networking | Storage | Tags

Root device details

Root device name /dev/sda1	Root device type EBS	EBS optimization enabled
-------------------------------	-------------------------	-----------------------------

Block devices

Volume ID	Device name	Volume size (GiB)	Volume State	Attachment status	EBS card index	Attachment time
vol-0eca844fe3bef18a1	/dev/sda1	8	In-use	Attached	0	2026/08/04 05
vol-026665b0f18f1e10e	/dev/sdh	2	In-use	Attached	0	2026/08/04 05

Volume monitoring (1)

The screenshot shows the AWS Management Console interface for creating a new EBS volume. The breadcrumb navigation at the top indicates the path: EC2 > Volumes > Create volume. The main heading is 'Create volume' with an 'Info' link. Below this, a sub-heading 'Volume settings' is followed by a description: 'Create an Amazon EBS volume to attach to any EC2 instance in the same Availability Zone.' The settings are organized into sections: 'Volume type' (General Purpose SSD (gp3)), 'Size (GiB)' (2, with a range of 1 to 65536 GiB), 'IOPS' (3000, with a range of 3000 to 80000 IOPS), 'Throughput (MiB/s)' (125, with a range of 125 to 2000 MiB/s), and 'Availability Zone' (aps1-az1 (ap-south-1a)). Each section has an 'Info' link for more details.

**3. Log in to your EC2 instance using SSH and format the attached EBS volume, then mount it to a new directory named `/mnt/mydata`.
Hint: Use `lsblk` to find your volume, `mkfs` to format, and `mount` to attach it to the directory.**

Answer

The EC2 instance was accessed using SSH. The attached EBS volume was identified, formatted with the **ext4** file system, and mounted to the `/mnt/mydata` directory.

Commands Used

```
lsblk
sudo mkfs -t ext4 /dev/xvdf
sudo mkdir -p /mnt/mydata
sudo mount /dev/xvdf /mnt/mydata
df -h
```

Note: On some EC2 instances, the EBS volume may appear as `/dev/nvme1n1` instead of `/dev/xvdf`. Use the device name shown by the `lsblk` command.

Steps Performed

1. Connected to the EC2 instance using SSH.
2. Used the `lsblk` command to identify the attached EBS volume.
3. Formatted the volume with the ext4 file system.
4. Created the mount point `/mnt/mydata`.

5. Mounted the EBS volume to /mnt/mydata.
6. Verified the successful mount using the df -h command.

Result

The newly attached **2 GB Amazon EBS** volume was successfully formatted and mounted at **/mnt/mydata**, making it available for storing application data and files.

```

ubuntu@ip-172-31-45-216: ~ × + v
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\Shreya> cd Downloads
PS C:\Users\Shreya\Downloads>
PS C:\Users\Shreya\Downloads> ssh -i "admin@123.pem" ubuntu@ec2-13-201-115-43.ap-south-1.compute.amazonaws.com
The authenticity of host 'ec2-13-201-115-43.ap-south-1.compute.amazonaws.com (13.201.115.43)' can't be established.
ED25519 key fingerprint is SHA256:Rfw11daK6hY+S10FpgffKC4GQMX6/V2FVhXZTid2rwg.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added 'ec2-13-201-115-43.ap-south-1.compute.amazonaws.com' (ED25519) to the list of known hosts.
Welcome to Ubuntu 26.04 LTS (GNU/Linux 7.0.0-1006-aws x86_64)

 * Documentation:  https://docs.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Tue Aug  4 03:43:35 UTC 2026

System load:  0.0           Temperature:   -273.1 C
Usage of /:   30.5% of 6.61GB Processes:      116
Memory usage: 26%          Users logged in: 0
Swap usage:   0%           IPv4 address for ens5: 172.31.45.216

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

ubuntu@ip-172-31-45-216:~$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
loop0        7:0      0   74M  1 loop /snap/core22/2411
loop1        7:1      0  28.2M  1 loop /snap/amazon-ssm-agent/13009
loop2        7:2      0  49.3M  1 loop /snap/snapd/26865
nvme0n1     259:0    0    8G  0 disk
├─nvme0n1p1 259:1    0   6.9G  0 part /
├─nvme0n1p13 259:2    0 1023M  0 part /boot
├─nvme0n1p14 259:3    0    4M  0 part
└─nvme0n1p15 259:4    0 106M  0 part /boot/efi
nvme1n1     259:5    0    2G  0 disk
ubuntu@ip-172-31-45-216:~$ sudo mkfs -t ext4 /dev/nvme1n1
mke2fs 1.47.2 (1-Jan-2025)
Creating filesystem with 524288 4k blocks and 131072 inodes

```

```

ubuntu@ip-172-31-45-216: ~
└─nvme0n1p15 259:4 0 106M 0 part /boot/efi
nvme1n1 259:5 0 2G 0 disk
ubuntu@ip-172-31-45-216:~$ sudo mkfs -t ext4 /dev/nvme1n1
mke2fs 1.47.2 (1-Jan-2025)
Creating filesystem with 524288 4k blocks and 131072 inodes
Filesystem UUID: 7720da34-1e75-4f88-a24e-c292953a0143
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912

Allocating group tables: done
Writing inode tables: done
Creating journal (16384 blocks): done
Writing superblocks and filesystem accounting information: done

ubuntu@ip-172-31-45-216:~$ sudo mkdir -p /mnt/mydata
ubuntu@ip-172-31-45-216:~$ sudo mount /dev/nvme1n1 /mnt/mydata
ubuntu@ip-172-31-45-216:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        6.7G  2.1G  4.6G  31% /
tmpfs            455M    0  455M   0% /dev/shm
tmpfs            182M  900K  181M   1% /run
efivarfs         128K   3.1K  120K   3% /sys/firmware/efi/efivars
tmpfs            455M    0  455M   0% /tmp
none             1.0M    0   1.0M   0% /run/credentials/systemd-journald.service
none             1.0M    0   1.0M   0% /run/credentials/systemd-resolved.service
/dev/nvme0n1p13  989M   96M  827M  11% /boot
/dev/nvme0n1p15  105M   6.3M   99M   7% /boot/efi
none             1.0M    0   1.0M   0% /run/credentials/systemd-networkd.service
none             1.0M    0   1.0M   0% /run/credentials/serial-getty@ttyS0.service
none             1.0M    0   1.0M   0% /run/credentials/getty@tty1.service
tmpfs            91M   8.0K   91M   1% /run/user/1000
/dev/nvme1n1     2.0G  536K  1.8G   1% /mnt/mydata
ubuntu@ip-172-31-45-216:~$

```

4. Simulate a Zomato-style image upload: create a text file named `zomato_menu.txt` on your EC2 instance, copy it to your mounted EBS volume, and verify it is accessible from the new mount point.

Answer

A sample file named `zomato_menu.txt` was created on the EC2 instance to simulate a restaurant menu. The file was then copied to the mounted Amazon EBS volume at `/mnt/mydata` and verified successfully.

Commands Used

```

cd
touch zomato_menu.txt
echo "Masala Dosa - ₹120
Paneer Pizza - ₹250
Veg Burger - ₹180" > zomato_menu.txt
cat zomato_menu.txt
sudo cp zomato_menu.txt /mnt/mydata/
ls -l /mnt/mydata
cat /mnt/mydata/zomato_menu.txt

```

Steps Performed

1. Logged in to the EC2 instance using SSH.
2. Created a file named `zomato_menu.txt`.
3. Added sample Zomato menu content to the file.
4. Verified the file content using the `cat` command.
5. Copied the file to the mounted EBS volume located at `/mnt/mydata`.
6. Verified that the file existed in the mounted directory.
7. Opened the copied file from the mounted EBS volume to confirm it was accessible.

Result

The `zomato_menu.txt` file was successfully copied to the mounted Amazon EBS volume and was accessible from `/mnt/mydata`, demonstrating that the EBS volume was functioning correctly as persistent storage.

```
ubuntu@ip-172-31-45-216:~$  
ubuntu@ip-172-31-45-216:~$  
ubuntu@ip-172-31-45-216:~$ cd  
ubuntu@ip-172-31-45-216:~$ touch zomato_menu.txt  
ubuntu@ip-172-31-45-216:~$ echo "Masala Dosa - ₹120  
Paneer Pizza - ₹250  
Veg Burger - ₹180" > zomato_menu.txt  
ubuntu@ip-172-31-45-216:~$ cat zomato_menu.txt  
Masala Dosa - ₹120  
Paneer Pizza - ₹250  
Veg Burger - ₹180  
ubuntu@ip-172-31-45-216:~$ sudo cp zomato_menu.txt /mnt/mydata/  
ubuntu@ip-172-31-45-216:~$ ls -l /mnt/mydata  
total 20  
drwx----- 2 root root 16384 Aug  4 03:44 lost+found  
-rw-r--r-- 1 root root    63 Aug  4 03:49 zomato_menu.txt  
ubuntu@ip-172-31-45-216:~$ cat /mnt/mydata/zomato_menu.txt  
Masala Dosa - ₹120  
Paneer Pizza - ₹250  
Veg Burger - ₹180  
ubuntu@ip-172-31-45-216:~$
```

5. Detach the EBS volume from your EC2 instance using the AWS Console, then reattach it to the same or a different instance. Describe what happens to the data you stored and why this behavior is useful for apps like Flipkart or Myntra order tracking.

Answer

The attached Amazon EBS volume was successfully detached from the EC2 instance using the AWS Management Console and then reattached to the same EC2 instance. After mounting the volume again, the previously stored file `zomato_menu.txt` was still present and accessible.

Steps Performed

1. Logged in to the AWS Management Console.
2. Opened the EC2 service.
3. Navigated to Elastic Block Store → Volumes.
4. Selected the attached 2 GB EBS volume.
5. Clicked Actions → Detach Volume.
6. Waited until the volume status changed to Available.
7. Clicked Actions → Attach Volume.
8. Selected the EC2 instance and attached the volume using the device name `/dev/sdf`.
9. Logged in to the EC2 instance using SSH.
10. Mounted the EBS volume to `/mnt/mydata`.
11. Verified that `zomato_menu.txt` was still available.

Observation

The data stored on the Amazon EBS volume **remained intact** even after detaching and reattaching the volume. This is because **Amazon EBS provides persistent block storage**, meaning data is stored independently of the EC2 instance and is not deleted when the volume is detached.

Real-World Use Case

This behavior is very useful for e-commerce applications such as **Flipkart** or **Myntra**. If an application server fails or needs to be replaced, the existing EBS volume containing important information such as **order history, customer records, or product data** can simply be attached to another EC2 instance. This allows the application to continue using the same data without loss, improving reliability and minimizing downtime.

Result

The EBS volume was successfully detached and reattached, and all previously stored data remained available. This demonstrates the persistent nature of Amazon EBS storage and its importance for applications that require reliable and durable data storage.

